Aug 5, 2025

# How to run gpt-oss with Transformers

 Dominik Kundel (OpenAI)

Open in Github

The Transformers library by Hugging Face provides a flexible way to load and run large language models locally or on a server. This guide will walk you through running OpenAI gpt-oss-20b or OpenAI gpt-oss-120b using Transformers, either with a high-level pipeline or via low-level `generate` calls with raw token IDs.

We'll cover the use of OpenAI gpt-oss-20b or OpenAI gpt-oss-120b with the high-level pipeline abstraction, low-level `generate` calls, and serving models locally with `transformers serve`, with in a way compatible with the Responses API.

In this guide we'll run through various optimised ways to run the **gpt-oss models via Transformers.**

Bonus: You can also fine-tune models via transformers, check out our fine-tuning guide here.

## Pick your model

Both **gpt-oss** models are available on Hugging Face:

- `openai/gpt-oss-20b`
  - ~16GB VRAM requirement when using MXFP4
  - Great for single high-end consumer GPUs
- `openai/gpt-oss-120b`
  - Requires ≥60GB VRAM or multi-GPU setup
  - Ideal for H100-class hardware

Both are **MXFP4 quantized** by default. Please, note that MXFP4 is supported in Hopper or later architectures. This includes data center GPUs such as H100 or GB200, as well as the



the 20b parameter model).

## Quick setup

1. **Install dependencies**
   It's recommended to create a fresh Python environment. Install transformers, accelerate, as well as the Triton kernels for MXFP4 compatibility:

```
pip install -U transformers accelerate torch triton kernels pip install git+https://github.com/
```

2. **(Optional) Enable multi-GPU**
   If you're running large models, use Accelerate or torchrun to handle device mapping automatically.

## Create an Open AI Responses / Chat Completions endpoint

To launch a server, simply use the `transformers serve` CLI command:

```
transformers serve
```

The simplest way to interact with the server is through the transformers chat CLI

```
transformers chat localhost:8000 --model-name-or-path openai/gpt-oss-20b
```

or by sending an HTTP request with cURL, e.g.

```
curl -X POST http://localhost:8000/v1/responses -H "Content-Type: application/json" -d '{"messa
```

Additional use cases, like integrating `transformers serve` with Cursor and other tools, are detailed in the documentation.

## Quick inference with pipeline

The easiest way to run the gpt-oss models is with the Transformers high-level `pipeline` API:

```
from transformers import pipeline

generator = pipeline(
    "text-generation",
    model="openai/gpt-oss-20b",
    torch_dtype="auto",
    device_map="auto"  # Automatically place on available GPUs
)

messages = [
    {"role": "user", "content": "Explain what MXFP4 quantization is."},
]

result = generator(
    messages,
    max_new_tokens=200,
    temperature=1.0,
)

print(result[0]["generated_text"])
```

## Advanced inference with `.generate()`

If you want more control, you can load the model and tokenizer manually and invoke the `.generate()` method:

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model_name = "openai/gpt-oss-20b"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype="auto",
    device_map="auto"
)

messages = [
    {"role": "user", "content": "Explain what MXFP4 quantization is."},
]
```

```
inputs = tokenizer.apply_chat_template(
    messages,
    add_generation_prompt=True,
    return_tensors="pt",
    return_dict=True,
).to(model.device)

outputs = model.generate(
    **inputs,
    max_new_tokens=200,
    temperature=0.7
)

print(tokenizer.decode(outputs[0]))
```

## Chat template and tool calling

OpenAI gpt-oss models use the harmony response format for structuring messages, including reasoning and tool calls.

To construct prompts you can use the built-in chat template of Transformers. Alternatively, you can install and use the openai-harmony library for more control.

To use the chat template:

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model_name = "openai/gpt-oss-20b"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    device_map="auto",
    torch_dtype="auto",
)

messages = [
    {"role": "system", "content": "Always respond in riddles"},
    {"role": "user", "content": "What is the weather like in Madrid?"},
]

inputs = tokenizer.apply_chat_template(
    messages,
    add_generation_prompt=True,
    return_tensors="pt",
```

```
    return_dict=True,
).to(model.device)

generated = model.generate(**inputs, max_new_tokens=100)
print(tokenizer.decode(generated[0][inputs["input_ids"].shape[-1] :]))
```

To integrate the `openai-harmony` library to prepare prompts and parse responses, first install it like this:

```
pip install openai-harmony
```

Here's an example of how to use the library to build your prompts and encode them to tokens:

```
import json
from openai_harmony import (
    HarmonyEncodingName,
    load_harmony_encoding,
    Conversation,
    Message,
    Role,
    SystemContent,
    DeveloperContent
)
from transformers import AutoModelForCausalLM, AutoTokenizer

encoding = load_harmony_encoding(HarmonyEncodingName.HARMONY_GPT_OSS)

# Build conversation
convo = Conversation.from_messages([
    Message.from_role_and_content(Role.SYSTEM, SystemContent.new()),
    Message.from_role_and_content(
        Role.DEVELOPER,
        DeveloperContent.new().with_instructions("Always respond in riddles")
    ),
    Message.from_role_and_content(Role.USER, "What is the weather like in SF?")
])

# Render prompt
prefill_ids = encoding.render_conversation_for_completion(convo, Role.ASSISTANT)
stop_token_ids = encoding.stop_tokens_for_assistant_action()

# Load model
model_name = "openai/gpt-oss-20b"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name, torch_dtype="auto", device_map="auto")
```

```
# Generate
outputs = model.generate(
    input_ids=[prefill_ids],
    max_new_tokens=128,
    eos_token_id=stop_token_ids
)

# Parse completion tokens
completion_ids = outputs[0][len(prefill_ids):]
entries = encoding.parse_messages_from_completion_tokens(completion_ids, Role.ASSISTANT)

for message in entries:
    print(json.dumps(message.to_dict(), indent=2))
```

Note that the `Developer` role in Harmony maps to the `system` prompt in the chat template.

## Multi-GPU & distributed inference

The large gpt-oss-120b fits on a single H100 GPU when using MXFP4. If you want to run it on multiple GPUs, you can:

- Use `tp_plan="auto"` for automatic placement and tensor parallelism
- Launch with `accelerate launch or torchrun` for distributed setups
- Leverage Expert Parallelism
- Use specialised Flash attention kernels for faster inference

```
from transformers import AutoModelForCausalLM, AutoTokenizer
from transformers.distributed import DistributedConfig
import torch

model_path = "openai/gpt-oss-120b"
tokenizer = AutoTokenizer.from_pretrained(model_path, padding_side="left")

device_map = {
    # Enable Expert Parallelism
    "distributed_config": DistributedConfig(enable_expert_parallel=1),
    # Enable Tensor Parallelism
    "tp_plan": "auto",
}

model = AutoModelForCausalLM.from_pretrained(
```

```
    model_path,
    torch_dtype="auto",
    attn_implementation="kernels-community/vllm-flash-attn3",
    **device_map,
)

messages = [
     {"role": "user", "content": "Explain how expert parallelism works in large language models
]

inputs = tokenizer.apply_chat_template(
    messages,
    add_generation_prompt=True,
    return_tensors="pt",
    return_dict=True,
).to(model.device)

outputs = model.generate(**inputs, max_new_tokens=1000)

# Decode and print
response = tokenizer.decode(outputs[0])
print("Model response:", response.split("<|channel|>final<|message|>")[-1].strip())
```

You can then run this on a node with four GPUs via

```
torchrun --nproc_per_node=4 generate.py
```